

Modelos de Regresión

Semana 9

Agenda

1. **Modelos de Regresión** — tipos, supuestos y variantes
2. **Métricas para Regresión** — MAE, MSE, RMSE, R^2 y análisis de residuos
3. **Regresión con Spark MLlib** — VectorAssembler, Pipeline y evaluación distribuida

PARTE 1

Modelos de Regresión

Tipos, supuestos y variantes principales

¿Qué es la Regresión?

La **regresión** es una tarea de **aprendizaje supervisado** cuya salida es un valor **continuo**.

Objetivo

- Aprender una función f tal que: $\hat{y} = f(x)$
- Minimizar la diferencia entre $\hat{y}$ e y

Diferencia con Clasificación

- Clasificación → etiqueta discreta (`spam`, `no spam`)
- Regresión → valor numérico (`$150,000`, `23.5°C`)

Ejemplos de uso

- Predicción de precios de vivienda
- Estimación de temperatura
- Proyección de ventas
- Tiempo de procesamiento de consultas SQL

Regresión Lineal Simple

Modela la relación entre una variable independiente x y una variable dependiente y :

$$\hat{y} = \beta_0 + \beta_1 x$$

Función de costo (MSE):

$$J(\beta_0, \beta_1) = (1/n) \sum (y_i - \hat{y}_i)^2$$

Mínimos Cuadrados Ordinarios (OLS): minimiza J con solución analítica cerrada:

$$\beta_1 = \sum (x_i - \bar{x})(y_i - \bar{y}) / \sum (x_i - \bar{x})^2 \quad \beta_0 = \bar{y} - \beta_1 \bar{x}$$

OLS garantiza el **mejor estimador lineal no sesgado** (teorema de Gauss-Markov) cuando se cumplen los supuestos del modelo.

(Built In, 2024a; Vignesh, 2023)

Regresión Lineal Múltiple

Extiende la regresión simple a **p features**:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p$$

Forma matricial: $\hat{y} = X\beta \rightarrow \hat{\beta} = (X^T X)^{-1} X^T y$

Cinco supuestos (Gauss-Markov):

#	Supuesto	Consecuencia de violación
1	Linealidad	Sesgo en estimadores
2	Independencia de errores	Autocorrelación
3	Homocedasticidad	Errores estándar inválidos
4	Normalidad de errores	Intervalos de confianza incorrectos
5	Ausencia de multicolinealidad	Coeficientes inestables

(Statology, 2024; Towards Data Science, 2023)

Regresión Polinomial

Cuando la relación entre `x` e `y` **no es lineal**, se elevan las features a potencias:

$$\hat{y} = \beta_0 + \beta_1 x + \beta_2 x^2 + \beta_3 x^3 + \dots$$

Ventaja

- Captura relaciones no lineales
- Sigue siendo lineal en los parámetros → OLS aplica
- Simple de implementar con

`PolynomialFeatures`

Riesgo

- Grado alto → **overfitting**
- Grado bajo → **underfitting**
- Selección de grado: validación cruzada

Un polinomio de grado n pasa exactamente por $n+1$ puntos, pero generaliza muy mal a datos nuevos. Usar regularización o CV para controlar la complejidad.

(Mithani, 2023)

Regularización: Motivación

Los modelos lineales complejos o con muchas features tienden al **overfitting**.

Solución: agregar un término de penalización al costo:

$$J_{\text{reg}}(\beta) = J_{\text{OLS}}(\beta) + \lambda \cdot \Omega(\beta)$$

Término	Nombre	Efecto
$\lambda \ \beta\ _2^2$	L2 / Ridge	Reduce magnitud de coeficientes
$\lambda \ \beta\ _1$	L1 / Lasso	Fuerza coeficientes a cero
$\lambda [\alpha \ \beta\ _1 + (1-\alpha) \ \beta\ _2^2]$	Elastic Net	Combinación de ambos

λ controla la fuerza de regularización:

- $\lambda \rightarrow 0$: sin regularización (OLS)
- $\lambda \rightarrow \infty$: coeficientes $\rightarrow 0$

(Brenndoerfer, 2024)

Ridge Regression (Regularización L2)

Función de costo:

$$J_{\text{Ridge}} = \sum (y_i - \hat{y}_i)^2 + \lambda \sum \beta_j^2$$

- Solución analítica: $\hat{\beta} = (X^T X + \lambda I)^{-1} X^T y$
- **Shrinkage**: todos los coeficientes se reducen, pero **ninguno llega a cero**
- Ideal con **multicolinealidad** o muchas features correlacionadas

Mayor λ

- Mayor sesgo
- Menor varianza
- Coeficientes más pequeños

Menor λ

- Menor sesgo
- Mayor varianza
- Se acerca a OLS

Lasso Regression (Regularización L1)

Función de costo:

$$J_{\text{Lasso}} = \sum (y_i - \hat{y}_i)^2 + \lambda \sum |\beta_j|$$

- **Sparsity**: fuerza algunos coeficientes exactamente a **cero** → selección automática de features
- No tiene solución analítica cerrada (usa descenso de coordenadas)
- Ideal para datasets de **alta dimensionalidad** con pocas features relevantes

¿Por qué L1 produce ceros y L2 no?

La región de restricción L1 (diamante) tiene esquinas sobre los ejes. El óptimo suele tocar una esquina → coeficiente = 0. L2 (esfera) no tiene esquinas.

Elastic Net

Combina L1 y L2:

$$J_{\text{EN}} = \sum (y_i - \hat{y}_i)^2 + \lambda [\alpha \|\beta\|_1 + (1-\alpha) \|\beta\|_2^2]$$

- $\alpha = 1$: Lasso puro
- $\alpha = 0$: Ridge puro
- $0 < \alpha < 1$: Elastic Net

Cuándo usar Elastic Net

- Features en grupos correlacionados
- Se desea selección de features (Lasso) con más estabilidad (Ridge)
- Datasets con $p \gg n$

Hiperparámetros

- λ : fuerza total de regularización
- α : ratio L1 / L2
- Seleccionar vía cross-validation

Comparativa: OLS, Ridge, Lasso y Elastic Net

Método	Penalización	Selección de features	Multicolinealidad	Cuándo usar
OLS	Ninguna	No	Problema	Datos limpios, pocas features
Ridge	$L2 \ \beta\ ^2$	No	Maneja bien	Features correlacionadas
Lasso	$L1 \ \beta\ _1$	Sí	Elige 1 del grupo	Alta dimensionalidad, pocas relevantes
Elastic Net	$L1 + L2$	Sí	Maneja bien	Grupos correlacionados + selección

(Brenndoerfer, 2024; Bhuva, 2024; scikit-learn, 2024a, 2024b)

Decision Tree y Random Forest para Regresión

Decision Tree Regressor

- Divide el espacio de features recursivamente
- Predicción: **media** de los valores en la hoja
- Pros: interpretable, sin supuestos de linealidad
- Contras: alta varianza, overfitting fácil

Random Forest Regressor

- **Ensemble** de árboles sobre subsets aleatorios (**bagging**)
- Predicción: **promedio** de todos los árboles
- Reduce varianza sin aumentar sesgo significativamente
- Robusto a outliers, no requiere escalado de features

Random Forest es uno de los algoritmos más robustos *out-of-the-box* para regresión.

Clave tunear `n_estimators` y `max_depth`.

(Oracle, 2024; Brenndoerfer, 2024b)

Gradient Boosting para Regresión

Construcción **secuencial** de árboles: cada árbol corrige los errores del anterior.

Algoritmo:

1. Inicializar $F_0(x) = \bar{y}$

2. Para $t = 1, \dots, T$:

- Calcular residuos: $r_i = y_i - F_{t-1}(x_i)$
- Ajustar árbol h_t a los residuos
- Actualizar: $F_t(x) = F_{t-1}(x) + \eta \cdot h_t(x)$

Hiperparámetros clave:

Parámetro	Efecto
<code>n_estimators</code>	Número de árboles (más = mejor, pero más lento)
<code>learning_rate</code> (η)	Tamaño del paso — compensar con más árboles
<code>max_depth</code>	Complejidad de cada árbol

(DataCamp, 2024a)

Support Vector Regression (SVR)

Busca un hiperplano dentro de un **tubo de tolerancia** ϵ que contenga la mayor cantidad de puntos.

Objetivo: minimizar $\|w\|^2$ sujeto a $|y_i - \hat{y}_i| \leq \epsilon$

Parámetros clave:

- **C**: penaliza puntos fuera del tubo (fuerza de regularización)
- **ϵ** : ancho del tubo de tolerancia
- **Kernel**: transforma features (`linear` , `rbf` , `poly`)

Ventajas

- Eficaz en alta dimensionalidad
- Robusto a outliers dentro del tubo
- Flexible vía kernels no lineales

(Analytics Vidhya, 2020)

Desventajas

- Escala mal con datasets grandes
- Requiere escalado de features
- Difícil de interpretar

Resumen: ¿Cuándo usar cada modelo?

Modelo	Interpretabilidad	Escalabilidad	No linealidad	Selección features
OLS / MLR	Alta	Media	No	No
Ridge	Media	Alta	No	No
Lasso / Elastic Net	Media	Alta	No	Sí
Decision Tree	Alta	Media	Sí	Implícita
Random Forest	Baja	Alta	Sí	Implícita
Gradient Boosting	Baja	Alta	Sí	Implícita
SVR	Muy baja	Baja	Sí (kernel)	No

PARTE 2

Métricas para Regresión

Cómo medir y comparar el desempeño de modelos

MAE — Mean Absolute Error

$$\text{MAE} = (1/n) \sum |y_i - \hat{y}_i|$$

Propiedades:

- Mismas unidades que la variable objetivo
- **Robusto a outliers**: penaliza proporcionalmente, sin elevar al cuadrado
- Interpretación directa: *"En promedio el modelo se equivoca en $\pm X$ unidades"*

Ventajas

- Fácil de interpretar
- Menos sensible a errores grandes
- Métrica mediana en distribuciones simétricas

(Mundal, 2024)

Desventajas

- No diferenciable en 0 (no ideal para gradient descent)
- No distingue entre muchos errores pequeños y pocos errores grandes

MSE y RMSE

MSE — Mean Squared Error

$$\text{MSE} = (1/n) \sum (y_i - \hat{y}_i)^2$$

- **Penaliza fuertemente** errores grandes (efecto cuadrático)
- Completamente diferenciable → ideal como función de pérdida en optimización
- Unidades: cuadrado de la variable objetivo (*menos interpretable*)

RMSE — Root Mean Squared Error

$$\text{RMSE} = \sqrt{\text{MSE}}$$

- **Mismas unidades** que la variable objetivo
- La métrica **más usada** en práctica para regresión
- Sensible a outliers (hereda de MSE)

$RMSE \geq MAE$ siempre. Cuanto mayor la diferencia entre ambas, mayor es la influencia de valores atípicos en el dataset.

(GeeksforGeeks, 2024b; Matalonga, 2024)

R² — Coeficiente de Determinación

$$R^2 = 1 - (SS_{\text{res}} / SS_{\text{tot}}) = 1 - [\Sigma (y_i - \hat{y}_i)^2 / \Sigma (y_i - \bar{y})^2]$$

Interpretación: proporción de la varianza total explicada por el modelo.

Valor	Significado
R² = 1	Ajuste perfecto — el modelo explica toda la varianza
R² = 0	El modelo equivale a predecir siempre $\bar{y}$
R² < 0	El modelo es peor que predecir la media

Trampa común: R² siempre aumenta al agregar features, aunque sean irrelevantes. Para comparar modelos con distinto número de predictores, usar **R² Ajustado**.

(Built In, 2024b; DataCamp, 2024b)

R² Ajustado

$$R^2_{\text{adj}} = 1 - \left[(1 - R^2) (n - 1) / (n - p - 1) \right]$$

Donde n = observaciones y p = número de features.

	R ²	R ² Ajustado
Agregar feature irrelevante	Siempre aumenta	Puede disminuir
Penaliza complejidad	No	Sí
Uso recomendado	Una sola configuración	Comparar modelos
Rango	$(-\infty, 1]$	$(-\infty, 1]$

R^2 Ajustado es el estándar para comparar modelos con distinto número de predictores.
Una caída al agregar una feature indica que no aporta valor real.

(AnalystPrep, 2024)

MAPE y RMSLE

MAPE — Mean Absolute Percentage Error

$$\text{MAPE} = (100/n) \sum | (y_i - \hat{y}_i) / y_i |$$

- Error **relativo** expresado en porcentaje → comunicable a stakeholders
- **Problema:** indefinido cuando $y_i = 0$; sesgado con valores pequeños

RMSLE — Root Mean Squared Log Error

$$\text{RMSLE} = \sqrt{ (1/n) \sum (\log(\hat{y}_i + 1) - \log(y_i + 1))^2 }$$

- Útil cuando el target tiene **distribución sesgada** (precios, salarios, tráfico web)
- Penaliza más la **subestimación** que la sobreestimación
- Popular en competencias Kaggle con targets con cola larga

(Arize AI, 2024; Towards Data Science, 2024)

Análisis de Residuos

Los residuos $e_i = y_i - \hat{y}_i$ deben examinarse para validar los supuestos del modelo.

Gráficos diagnósticos clave:

Gráfico	Qué detecta	Patrón problemático
Residuos vs Fitted	Linealidad, homocedasticidad	Curva o embudo
Q-Q Plot	Normalidad de errores	Desviación de la diagonal
Scale-Location	Homocedasticidad	Pendiente en la línea de tendencia
Residuos vs Leverage	Puntos influyentes	Puntos fuera de la distancia de Cook

Un buen modelo muestra residuos distribuidos **aleatoriamente alrededor de cero**, sin patrones sistemáticos ni heterocedasticidad.

(Six Sigma DSI, 2024; Qualtrics, 2024)

Tabla Comparativa de Métricas

Métrica	Unidades	Outliers	Interpretación	Cuándo usar
MAE	= y	Robusto	Error promedio absoluto	Interpretabilidad directa
MSE	y^2	Sensible	Error cuadrático medio	Loss function / optimización
RMSE	= y	Sensible	Error cuadrático medio (raíz)	Métrica estándar principal
R²	Sin unidad	Medio	% varianza explicada	Bondad de ajuste global
R²_adj	Sin unidad	Medio	% varianza (penaliza p)	Comparar modelos distintos
MAPE	%	Muy sensible	Error relativo %	Comunicación ejecutiva
RMSLE	Sin unidad	Medio	Error log-escalado	Targets sesgados

PARTE 3

Regresión con Spark MLlib

Pipeline distribuido end-to-end

Algoritmos de Regresión en Spark MLlib

Spark MLlib ofrece una suite completa de algoritmos de regresión escalables:

Algoritmo	Clase PySpark	Tipo
Regresión Lineal	<code>LinearRegression</code>	Lineal + regularización L1/L2/Elastic Net
Regresión Lineal Generalizada	<code>GeneralizedLinearRegression</code>	Familias: Gaussian, Poisson, Gamma
Árbol de Decisión	<code>DecisionTreeRegressor</code>	No lineal
Random Forest	<code>RandomForestRegressor</code>	Ensemble (bagging)
Gradient Boosted Trees	<code>GBTRegressor</code>	Ensemble (boosting)
Regresión Isotónica	<code>IsotonicRegression</code>	No paramétrica, monotónica

(Apache Software Foundation, 2024e; Sharma, 2024)

VectorAssembler: Ensamblado de Features

Paso **obligatorio** en MLlib: todas las features deben estar en un único vector columna.

```
from pyspark.ml.feature import VectorAssembler

assembler = VectorAssembler(
    inputCols=["rooms", "age", "distance", "income"],
    outputCol="features"
)

df_assembled = assembler.transform(df)
```

Resultado:

rooms	age	distance	income	features
4	10	2.5	55000	[4.0, 10.0, 2.5, 55000.0]

(Apache Software Foundation, 2024a)

Pipeline API: Transformers + Estimators

Un **Pipeline** encadena transformaciones y modelo en un objeto reproducible:

```
from pyspark.ml import Pipeline
from pyspark.ml.feature import VectorAssembler, StandardScaler
from pyspark.ml.regression import LinearRegression

assembler = VectorAssembler(inputCols=feature_cols, outputCol="raw_features")
scaler      = StandardScaler(inputCol="raw_features", outputCol="features")
lr          = LinearRegression(featuresCol="features", labelCol="price")

pipeline = Pipeline(stages=[assembler, scaler, lr])

model      = pipeline.fit(train_df)
predictions = model.transform(test_df)
```

El Pipeline garantiza que el **mismo preprocesamiento** se aplique en train y test — evitando data leakage. Es la forma recomendada para producción.

(33rd Square, 2024; Apache Software Foundation, 2024b)

LinearRegression: Regularización en Spark

```
from pyspark.ml.regression import LinearRegression

lr = LinearRegression(
    featuresCol="features",
    labelCol="price",
    maxIter=100,
    regParam=0.1,          #  $\lambda$ : fuerza de regularización
    elasticNetParam=0.0    #  $\alpha$ : 0.0 = Ridge, 1.0 = Lasso
)

model = lr.fit(train_df)

# Resumen del modelo entrenado
summary = model.summary
print(f"RMSE: {summary.rootMeanSquaredError:.4f}")
print(f"R²: {summary.r2:.4f}")
print(f"Coeficientes: {model.coefficients}")
print(f"Intercepto: {model.intercept:.4f}")
```

`regParam` = λ controla la penalización. `elasticNetParam` = α mezcla L1 y L2: **0 = Ridge puro, 1 = Lasso puro.**

(Apache Software Foundation, 2024e)

GBRegressor: Gradient Boosted Trees

```
from pyspark.ml.regression import GBRegressor

gbt = GBRegressor(
    featuresCol="features",
    labelCol="price",
    maxIter=50,      # número de árboles
    maxDepth=5,      # profundidad de cada árbol
    stepSize=0.1     # learning rate ( $\eta$ )
)

model = gbt.fit(train_df)
predictions = model.transform(test_df)
```

Cuándo usar GBT sobre LinearRegression:

- Relaciones **no lineales** entre features y target
- **Interacciones** entre variables
- Features de tipos mixtos (numéricas + categóricas codificadas)
- Mayor capacidad predictiva a costo de menor interpretabilidad

(Apache Software Foundation, 2024e)

RegressionEvaluator

```
from pyspark.ml.evaluation import RegressionEvaluator

evaluator = RegressionEvaluator(
    labelCol="price",
    predictionCol="prediction"
)

rmse = evaluator.setMetricName("rmse").evaluate(predictions)
mae   = evaluator.setMetricName("mae").evaluate(predictions)
r2    = evaluator.setMetricName("r2").evaluate(predictions)
mse   = evaluator.setMetricName("mse").evaluate(predictions)

print(f"RMSE: {rmse:.2f}")
print(f"MAE: {mae:.2f}")
print(f"R²: {r2:.4f}")
```

Métricas soportadas: `rmse` · `mse` · `r2` · `mae` · `var`

(Apache Software Foundation, 2024c)

Tuning: TrainValidationSplit y CrossValidator

```
from pyspark.ml.tuning import TrainValidationSplit, CrossValidator, ParamGridBuilder

paramGrid = (ParamGridBuilder()
    .addGrid(lr.regParam, [0.01, 0.1, 1.0])
    .addGrid(lr.elasticNetParam, [0.0, 0.5, 1.0])
    .build())

# Opción A: más rápido
tvs = TrainValidationSplit(
    estimator=pipeline, estimatorParamMaps=paramGrid,
    evaluator=evaluator, trainRatio=0.8)

# Opción B: más robusto
cv = CrossValidator(
    estimator=pipeline, estimatorParamMaps=paramGrid,
    evaluator=evaluator, numFolds=5)

best_model = cv.fit(train_df)
```

Guardado de Modelos e Integración con MLflow

```
import mlflow
import mlflow.spark

# Guardar con MLflow (tracking completo)
with mlflow.start_run():
    model = pipeline.fit(train_df)
    mlflow.log_param("regParam", 0.1)
    mlflow.log_param("elasticNetParam", 0.0)
    mlflow.log_metric("rmse", rmse)
    mlflow.log_metric("r2", r2)
    mlflow.spark.log_model(model, "spark-regression-model")

# Cargar modelo guardado
loaded = mlflow.spark.load_model("runs:/<run_id>/spark-regression-model")
```


Sin MLflow (método nativo Spark):

```
model.save("/models/regression_v1")  
from pyspark.ml import PipelineModel  
model = PipelineModel.load("/models/regression_v1")
```

(MLflow, 2024; Toumy, 2024)

Pipeline End-to-End: Ejemplo Completo

```
from pyspark.ml import Pipeline
from pyspark.ml.feature import VectorAssembler
from pyspark.ml.regression import LinearRegression
from pyspark.ml.evaluation import RegressionEvaluator

# 1. Carga de datos
df = spark.read.csv("housing.csv", header=True, inferSchema=True)
train_df, test_df = df.randomSplit([0.8, 0.2], seed=42)

# 2. Feature engineering
assembler = VectorAssembler(
    inputCols=["rooms", "age", "distance", "income"],
    outputCol="features")

# 3. Modelo con regularización Ridge
lr = LinearRegression(featuresCol="features", labelCol="price",
                      regParam=0.1, elasticNetParam=0.0)

# 4. Pipeline + entrenamiento
model = Pipeline(stages=[assembler, lr]).fit(train_df)

# 5. Evaluación
preds = model.transform(test_df)
ev = RegressionEvaluator(labelCol="price", predictionCol="prediction")
print(f"RMSE: {ev.setMetricName('rmse').evaluate(preds):.2f}")
print(f"R²: {ev.setMetricName('r2').evaluate(preds):.4f}")
```

Referencias

Analytics Vidhya. (2020). *Support vector regression tutorial for machine learning*.

<https://www.analyticsvidhya.com/blog/2020/03/support-vector-regression-tutorial-for-machine-learning/>

AnalystPrep. (2024). *Adjusted coefficient of determination*. <https://analystprep.com/study-notes/cfa-level-2/quantitative-method/adjusted-coefficient-determination/>

Apache Software Foundation. (2024a). *VectorAssembler — PySpark 4.1.2 documentation*. <https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.ml.feature.VectorAssembler.html>

Apache Software Foundation. (2024b). *Extracting, transforming and selecting features — Apache Spark 4.1.2*. <https://spark.apache.org/docs/latest/ml-features.html>

Apache Software Foundation. (2024c). *RegressionEvaluator — PySpark 4.1.1 documentation*.
<https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.ml.evaluation.RegressionEvaluator.html>

Apache Software Foundation. (2024d). *ML tuning: Model selection and hyperparameter tuning — Spark 4.1.2*. <https://spark.apache.org/docs/latest/ml-tuning.html>

Apache Software Foundation. (2024e). *Classification and regression — Apache Spark 4.1.1*.
<https://spark.apache.org/docs/latest/ml-classification-regression.html>

Arize AI. (2024). *Mean absolute percentage error (MAPE): What you need to know*.
<https://arize.com/blog-course/mean-absolute-percentage-error-mape-what-you-need-to-know/>

Bhuva, L. (2024). *Lasso regression: Unveiling the math behind feature selection and sparsity*. Medium. <https://medium.com/@lomashbhuva/lasso-regression-unveiling-the-math-behind-feature-selection-and-sparsity-e951f4cadabb>

Brenndoerfer, M. (2024a). *Ridge regression (L2 regularization): Complete guide with mathematical foundations*. <https://mbrenndoerfer.com/writing/ridge-regression-l2-regularization-complete-guide>

Brenndoerfer, M. (2024b). *Random forest: Complete guide to ensemble learning*. <https://mbrenndoerfer.com/writing/random-forest-ensemble-learning-bootstrap-sampling-feature-selection-classification-regression-guide>

Built In. (2024a). *Understanding ordinary least squares (OLS) regression*. <https://builtin.com/data-science/ols-regression>

Built In. (2024b). *R-squared and adjusted R-squared: Explained*. <https://builtin.com/data-science/adjusted-r-squared>

DataCamp. (2024a). *A guide to the gradient boosting algorithm*.
<https://www.datacamp.com/tutorial/guide-to-the-gradient-boosting-algorithm>

DataCamp. (2024b). *Coefficient of determination: What R-squared tells us*.
<https://www.datacamp.com/tutorial/coefficient-of-determination>

GeeksforGeeks. (2024a). *Linear regression in machine learning*.
<https://www.geeksforgeeks.org/machine-learning/ml-linear-regression/>

GeeksforGeeks. (2024b). *Mean squared error: Definition, formula, interpretation and examples*. <https://www.geeksforgeeks.org/maths/mean-squared-error/>

Jainindore, A. (2024). *Elastic net regression: Combined features of L1 and L2 regularization*. Medium. <https://medium.com/@abhishekjainindore24/elastic-net-regression-combined-features-of-l1-and-l2-regularization-6181a660c3a5>

Keith, J. (2021). *Handbook of regression modeling in people analytics*. <https://peopleanalytics-regression-book.org/linear-reg-ols.html>

Matalonga, H. (2024). *Choosing between MAE, MSE and RMSE*. <https://hmatalonga.com/blog/choosing-between-mae-mse-and-rmse/>

Meng, X., Bradley, J., Yavuz, B., Sparks, E., Venkataraman, S., Liu, D., Freeman, J., Tsai, D., Amde, M., Owen, S., Xin, D., Xin, R., Franklin, M. J., Zadeh, R., Zaharia, M., & Talwalkar, A. (2016). MLlib: Machine learning in Apache Spark. *Journal of Machine Learning Research*, 17(34), 1–7. <https://arxiv.org/pdf/1505.06807>

Mithani, D. (2023). *Overfitting and underfitting with polynomial regression*. Medium.
<https://medium.com/@danish.mithani/overfitting-and-underfitting-with-polynomial-regression-7a4ab2c8177a>

MLflow. (2024). *mlflow.spark*. MLflow documentation.
https://mlflow.org/docs/latest/python_api/mlflow.spark.html

Mundal, S. (2024). *Understanding MAE, MSE, and RMSE: Key metrics in machine learning*. DEV Community. https://dev.to/mondal_sabbha/understanding-mae-mse-and-rmse-key-metrics-in-machine-learning-4la2

Oracle. (2024). *Random forests, decision trees, and ensemble methods explained*. Oracle AI & Data Science Blog. <https://blogs.oracle.com/ai-and-datascience/post/random-forests-decision-trees-and-ensemble-methods-explained>

Qualtrics. (2024). *Interpreting residual plots to improve your regression*.

<https://www.qualtrics.com/support/stats-iq/analyses/regression-guides/interpreting-residual-plots-improve-regression/>

Runawayhorse001. (2024). *Regression — Learning Apache Spark with Python*.

<https://runawayhorse001.github.io/LearningApacheSpark/regression.html>

scikit-learn. (2024a). *Lasso — scikit-learn 1.9.0 documentation*. https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.Lasso.html

scikit-learn. (2024b). *ElasticNet — scikit-learn 1.9.0 documentation*. https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.ElasticNet.html

Sharma, S. (2024). *Machine learning with PySpark MLlib: Part 1 regression*. Medium.
<https://sharmashorya1996.medium.com/machine-learning-with-pyspark-mllib-part-1-regression-e7ad4d9780af>

Six Sigma DSI. (2024). *Residual analysis explained: Understand model fit & patterns*.
<https://sixsigmadsi.com/residual-analysis/>

Statology. (2024). *The five assumptions of multiple linear regression*.
<https://www.statology.org/multiple-linear-regression-assumptions/>

Toumy, H. (2024). *Saving and loading a model with, and without MLflow*. Spark Notes.
<https://haya-toumy.gitbook.io/spark-notes/pyspark/pyspark/related-to-ml/saving-and-loading-a-model-with-and-without-mlflow>

Towards Data Science. (2023). *Assumptions of multiple linear regression*.

<https://towardsdatascience.com/assumptions-of-multiple-linear-regression-d16f2eb8a2e7/>

Towards Data Science. (2024). *6 common metrics for your next regression project*.

<https://towardsdatascience.com/6-common-metrics-for-your-next-regression-project-4667cbc534a7/>

Vignesh, A. (2023). *Supervised linear regression using OLS model*. Medium.

<https://22vignesh97.medium.com/supervised-linear-regression-using-ols-model-d2561502343b>

Wikipedia. (2024). *Mean absolute error*. https://en.wikipedia.org/wiki/Mean_absolute_error

¡Gracias!